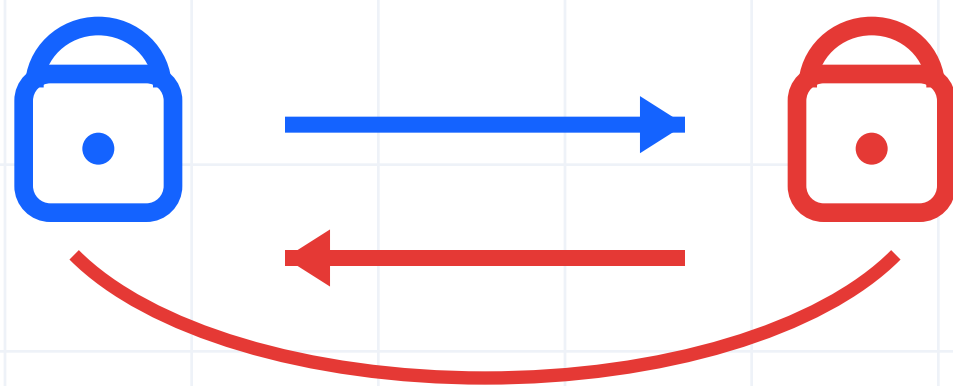

A deadlock isn't a bug.

It's two transactions doing
everything right,
in the wrong order.

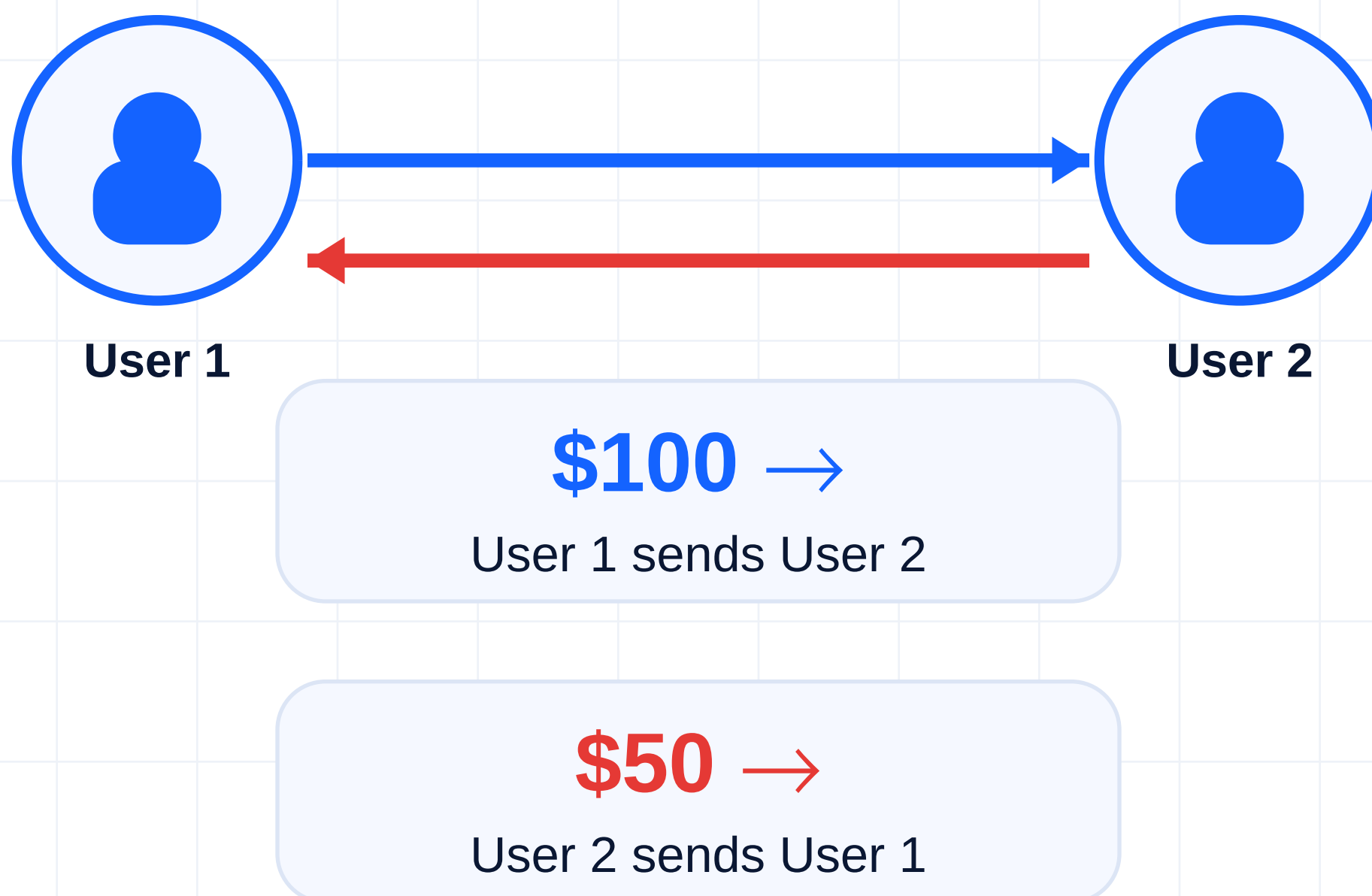


Concurrency looks simple.

Until two transactions need the same rows.

The setup

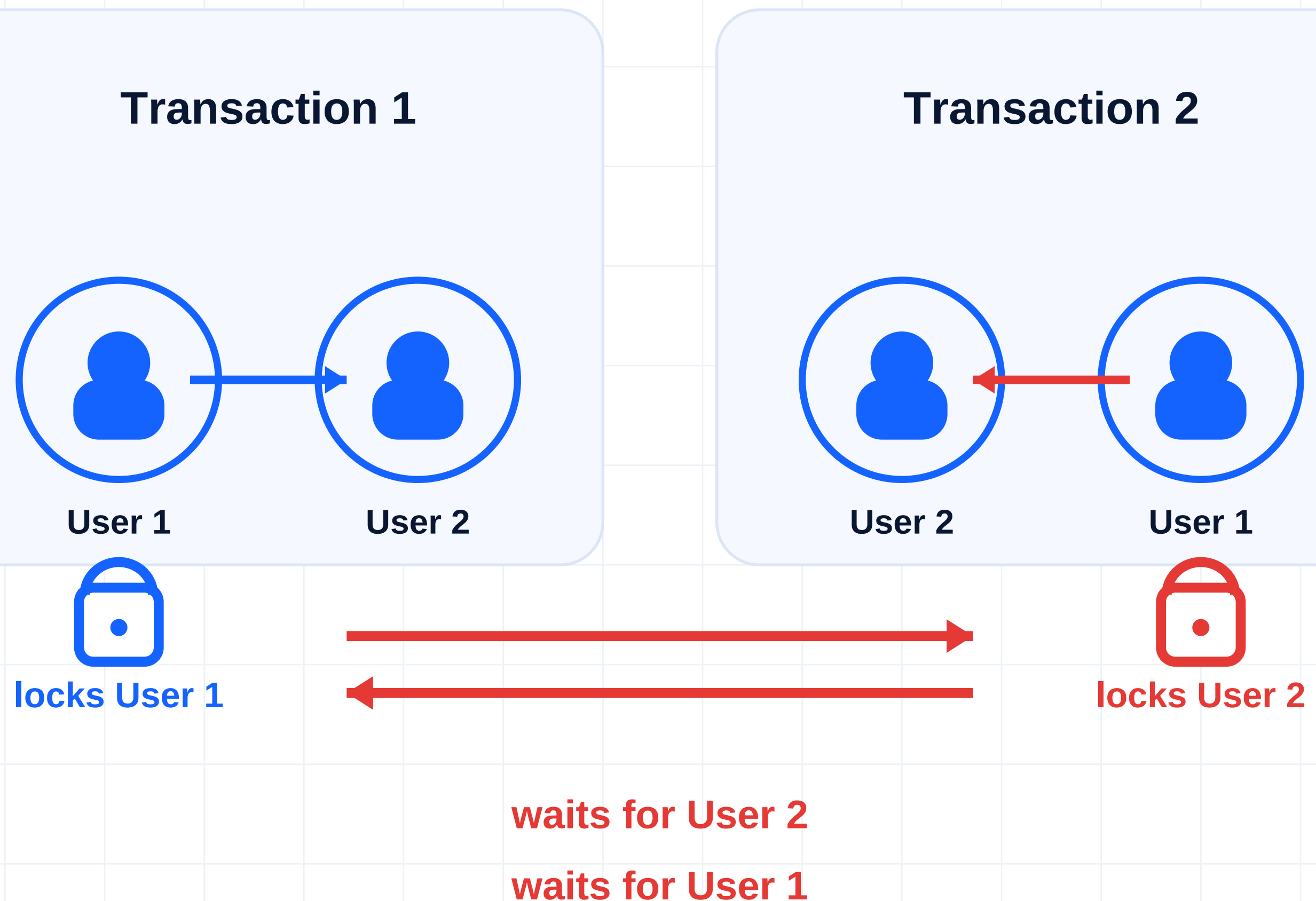
Two transfers happen at the same time.



Both transactions need to lock both accounts.

What happens

Each transaction locks one row first.



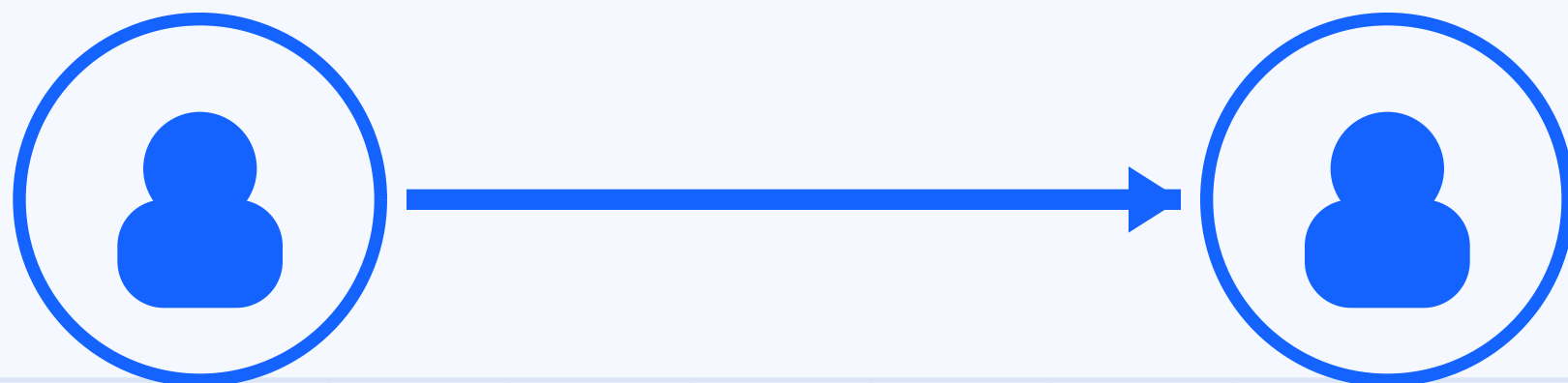
CIRCULAR WAIT

Neither transaction can move.

The fix: ordered locking

Use one consistent lock order.

Sort lock requests by a fixed key
account_id ascending



Account 1

Account 2

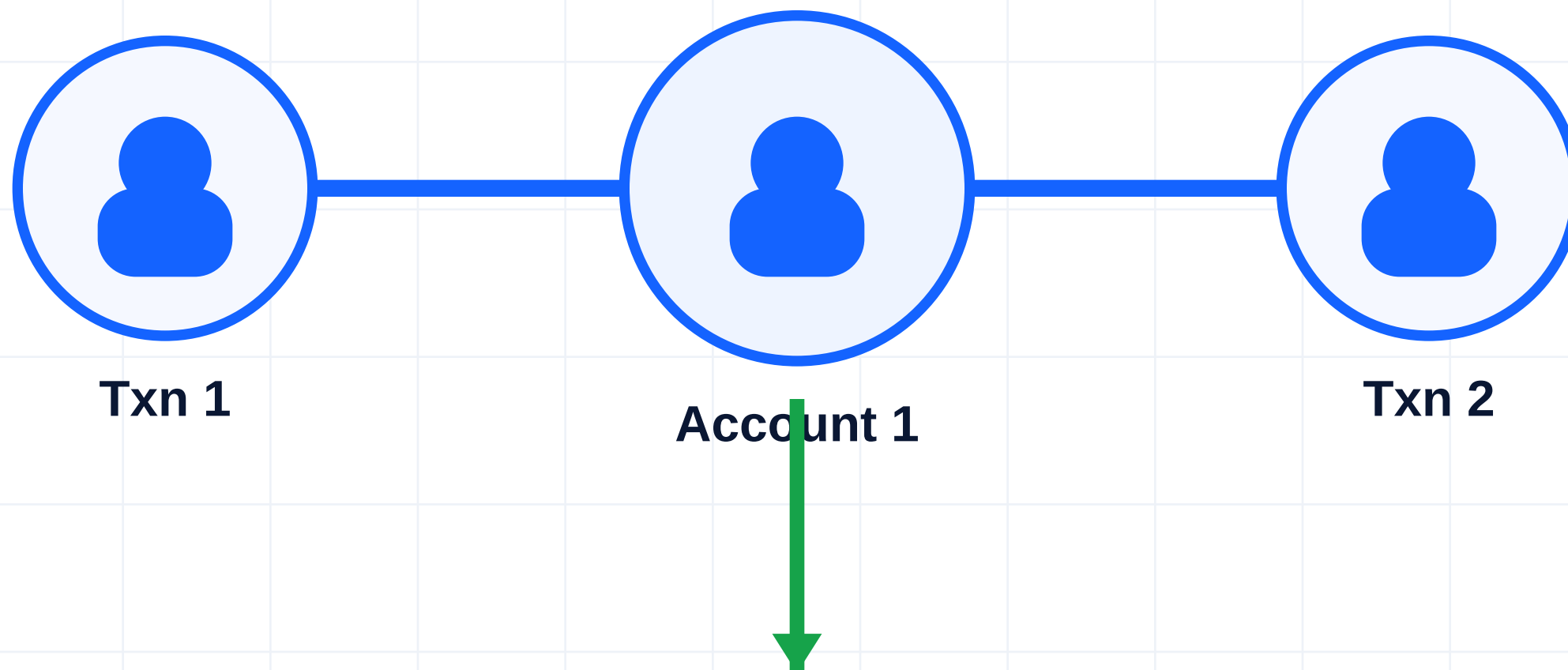
**Every transaction asks for Account 1 first.
Then it asks for Account 2.**

Same order → no circular wait

One transaction proceeds; the other queues.

Why it works

Both transactions contend for the same row first.



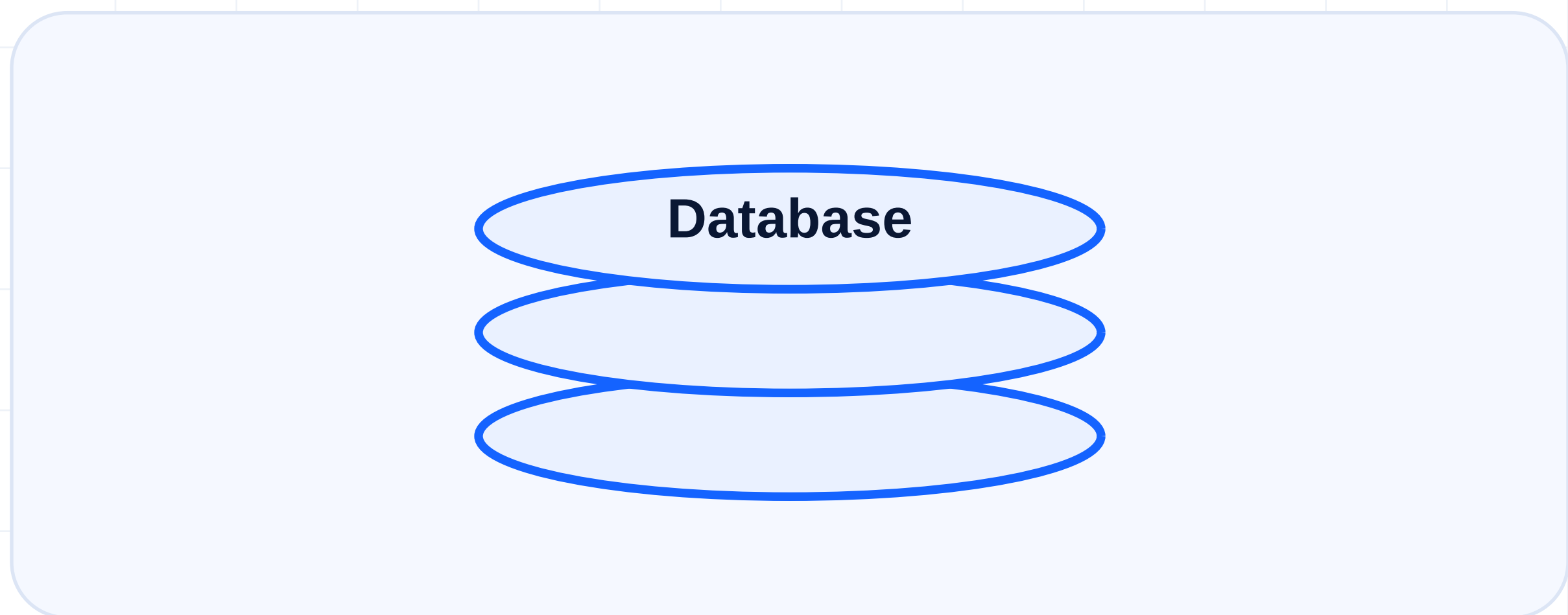
One transaction gets the lock.
The other waits — but there is no cycle.

No cycle can form

because everyone follows the same order.

Production reality check

Ordering prevents many deadlocks.



But databases also run deadlock detection.

If a cycle still forms

the database aborts one transaction with an error.

That error is expected to be handled.

The tip

Treat deadlocks as retryable.



Catch the specific
deadlock error
and retry the transaction.



Detect → Retry → Continue

Make the retry automatic.

Most concurrency bugs aren't about bigger locks.

They're about two processes
disagreeing on what order to ask for things.

Ever debugged a deadlock in production?

What did the actual root cause
turn out to be?

Share your experience in the comments.

